

# Early Warning of Hydrate Formation in Offshore Well Service Lines: A Comparison of XGBoost, TCN, and GRU at Matched False-Alarm Budgets

Mammadova, Aisel\*, Huseynov, Shamistan\*, Karimova, Rahima\*, Mammadova, Gulnur\*, Arabova, Saida\*

\*AI Academy, Baku, Azerbaijan

Team Enigma · DLE-AI-202, Cohort I 2026 · Track 2

{aysel.mamedova25, semistan.huseynov25, rahime.karimova25, saida.erebova25}@aiacademy.az,  
gulnur.mammedova25@aiacademy.edu.az

**Abstract**—Hydrate formation in offshore well service lines can cause costly shutdowns and safety hazards if not detected early. This study compares three models — XGBoost (feature-engineered baseline), a Temporal Convolutional Network (TCN), and a Gated Recurrent Unit (GRU) — for early warning of hydrate events using Petrobras’s 3W Dataset 2.0.0 (Event 9). All three models are evaluated at a matched false-alarm budget of one alarm per 100 operating hours, with lead time (time from first alarm to annotated transient onset) as the headline metric. Each model is trained under two conditions: real-only and real-plus-simulated data. We employ grouped cross-validation with well-level splits to prevent data leakage. Our results demonstrate that XGBoost trained exclusively on real data achieved the highest event recall of 23.3% ( $\pm 25.2\%$ ) and a mean lead time of 43.4 min across the two folds where any event was detected, at a matched false-alarm rate of 1.0%/h. Deep models (TCN and GRU) did not reliably detect events on the held-out test wells in any condition, primarily because only 14 real transient instances are available for training and evaluation. Simulated data did not improve performance for any model. The feature-engineered baseline remains the only approach that demonstrates non-trivial detection on this dataset.

**Reproduction:** all code, data processing, and evaluation are available at <https://github.com/AMammedova/team-hydrate3w>.

**Index Terms**—hydrate formation, early warning, time series classification, temporal convolutional network, gated recurrent unit, 3W dataset

## I. INTRODUCTION

Hydrate formation in subsea well service lines is a critical operational hazard in offshore oil and gas production. When gas hydrates crystallize inside flowlines they can cause complete blockages, requiring costly well shutdowns, intervention operations, and posing safety risks to personnel and equipment [15]. Early detection of the transient phase — the initial stage before full blockage — gives operators actionable lead time to apply thermodynamic or chemical inhibition before the line is lost.

The Petrobras 3W Dataset 2.0.0 [1] provides a rare public benchmark for this problem. Event 9 (hydrate formation in service lines) contains 57 real well instances, 150 simulated instances, and 594 normal-operation instances. However, a

careful inspection of the real instances reveals that **only 14 of the 57 real instances contain a transient phase**; only 3 reach full blockage. This makes Event 9 one of the most statistically challenging classification problems in the 3W collection: the positive class is not just rare, it is nearly absent in the test wells of any cross-validation fold.

This paper asks whether modern deep sequence models — a Temporal Convolutional Network (TCN) [2] and a Gated Recurrent Unit (GRU) [3] — can detect hydrate onset earlier than a feature-engineered XGBoost baseline [5], given the severe data scarcity. We evaluate all three models under two training conditions (real-only vs. real-plus-simulated) at a matched false-alarm budget of one alarm per 100 operating hours, using grouped cross-validation with well-level splits to prevent data leakage.

The remainder of this paper is organized as follows. Section II describes the 3W dataset and the measured properties of Event 9. Section III reviews related work. Section IV defines the models and feature extraction. Section V describes the cross-validation design and simulation policy. Section VI reports experimental results. Section VII discusses limitations, and Section VIII concludes.

## II. DATASET

### A. Dataset Overview

The 3W Dataset 2.0.0 [1] is a publicly available benchmark for undesirable event detection in oil wells. For Event 9 (hydrate formation in service lines), the dataset provides 57 real instances, 150 simulated instances, and 594 normal-operation instances, each a multivariate time series recorded at 1 Hz from up to 8 process sensors (pressures and temperatures along the wellbore and service line).

### B. Event-9 Properties

**Severity monotonicity.** All 57 real instances are monotonically increasing in severity: once a well enters the Transient phase it never returns to Normal, and once it enters the Established (blocked) phase it never returns to Transient. This

was verified programmatically. The property validates our label encoding as ordinal and justifies measuring lead time to the first irreversible state change.

**Scarcity of the positive class.** Only 14 of the 57 real instances contain a Transient phase; only 3 reach the Established (blockage) state. The transient phase is therefore our headline target: measuring lead time to blockage onset would cover only 3 events, an insufficient sample for any statistical claim. The 14 transient-phase instances span 7 distinct wells.

**Missingness and dead sensors.** NaN labels appear in all 57 real instances (median 11.2%, maximum 50.4% of samples in a channel). Three instances carry sensors that are completely dead for the entire recording (P-TPT and T-TPT frozen at constant values). These are detected by a frozen-sensor scan and the corresponding mask entries are set to 0 before windowing.

**Channel selection.** We evaluated two channel subsets: a 5-channel arm (P-PDG, P-TPT, T-TPT, P-MON-CKP, T-JUS-CKP) and a 7-channel arm that additionally includes annular sensors. All 14 transient events and all 3 blockage events survive both selections. The 5-channel arm was retained because the additional annular channels are absent in roughly 30% of real instances, which would inflate the missingness fraction without contributing coverage.

### C. Preprocessing

**Decimation and windowing.** Raw 1 Hz signals are decimated to one sample per 30 s, reducing noise without losing event-scale information. Windows of  $W = 60$  decimated samples (30 minutes) are extracted with a stride of 5 samples (150 s). The **median real instance** contains 32,135 timesteps at 1 Hz; at the decimated rate this yields roughly 1,071 windows per instance, well above the minimum needed to train a causal model.

**Labeling.** Each window is assigned the most severe label present in any of its 60 samples (`most_severe` rule). This is the correct choice: a window that ends in Established phase but whose first samples were still Transient must be reported as Established, because the last state is the one that matters operationally.

**Per-instance normalization.** Each channel is normalized using the mean and standard deviation of its *own* instance, computed over present samples only. This is mandatory, not cosmetic: the 15 hydrate wells and the 9 normal-operation wells are disjoint populations with no well in common. A model that sees raw absolute pressures could simply memorize which pressure range belongs to which population — per-instance normalization removes that shortcut from the value space. Sensor absence (the mask) remains and is treated separately.

**Cache statistics.** The built cache contains **42,259 real windows** from 364 usable real instances, of which 1,295 are Transient and 118 Established, giving a positive rate of 3.34%. The simulated arm adds 43,829 windows, of which 90.3% are positive — a distribution radically different from the real data.

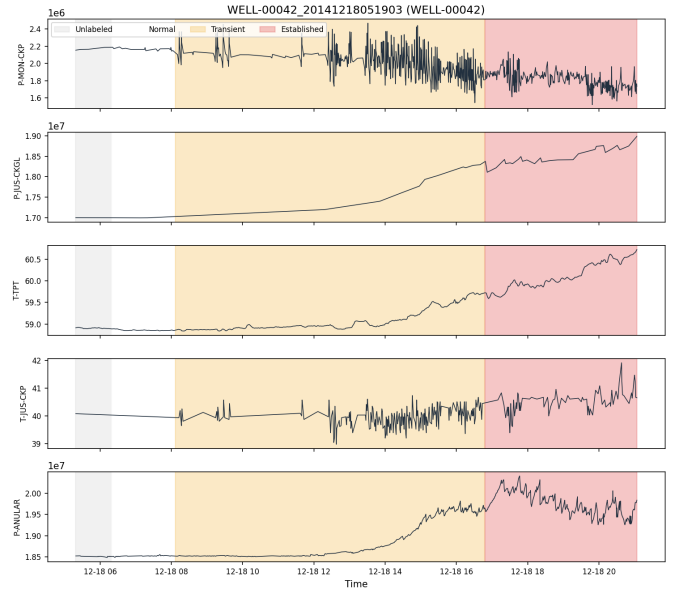


Fig. 1. Annotated multivariate sensor trace of a real Event-9 instance. Colour bands indicate operational regimes: Normal, Transient, and Established blockage. Early warning evaluates alarm lead time before the transient onset.

## III. RELATED WORK

### A. Sequence Models for Industrial Fault Detection

Temporal Convolutional Networks (TCN) [2] have demonstrated strong performance on a wide range of sequence classification benchmarks, matching or outperforming LSTM and GRU-based models while offering easier parallelization during training. The key design choice is the use of causal dilated convolutions, which expand the receptive field exponentially with depth without introducing future information into any prediction. This causal property is not merely a convenience: in a real early-warning system, any operation that looks beyond the current time step would inflate the reported lead time, making the system appear more capable than it is in deployment.

Gated Recurrent Units (GRU) [3] provide a simpler alternative to LSTM with fewer parameters. Like TCN, the GRU processes sequences step-by-step and is causal by construction when used in a unidirectional configuration. We use unidirectional GRU exclusively; a bidirectional GRU would introduce future context and invalidate the lead-time metric.

### B. Prior Work on the 3W Dataset

Vargas et al. [1] introduced the 3W dataset and demonstrated that classical machine learning methods (Random Forests, k-NN) achieve competitive performance on Event 9, but evaluated at the window level rather than the event level. Our evaluation protocol differs materially: we count *event recall* (did any alarm fire before the annotated transient onset?) and measure *lead time* (how early?), both of which are operationally meaningful quantities that window-level accuracy masks entirely. At a 3.34% positive rate, a classifier that labels

every window as Normal achieves 96.66% window accuracy while missing every event.

### C. Early Warning in Process Control

Process-industry early warning systems are reviewed in Qin [14], who notes that the dominant practical challenge is not algorithm design but the scarcity of labeled fault data: real process upsets are rare, expensive to induce, and often unrepeatable. This observation directly motivates our comparison of real-only vs. real-plus-simulated training conditions and our conservative evaluation protocol.

## IV. METHOD

### A. Baseline Feature Engineering

The baseline is treated as a real competitor rather than a formality: it receives the same folds, the same class-imbalance treatment and the same calibration step as the deep models, and a recorded tuning budget. A baseline that was never tuned would make the deep-model comparison uninterpretable.

1) *Input*: The baseline consumes the same cache as the deep models: channels-first `float32`  $[N, C, W]$  windows with an aligned `uint8` presence mask, where `mask = 1` marks a genuinely observed sample. With the main-arm channel set,  $C = 5$  and  $W = 60$  decimated samples at one sample per 30 s, so each window spans **30 minutes** with a stride of 150 s. On the built cache this gives **42,259 real windows** from 364 usable real instances, of which 1,295 are Transient and 118 Established — a positive rate of **3.34%**. All 14 transient events and all 3 blockage events survive the channel choice.

2) *Multi-timescale, mask-aware features*: For each window and channel we take three nested slices measured *from the end* of the window (the full 60 samples, the last 30, and the last 15) and compute six statistics on each: mean, standard deviation, minimum, maximum, least-squares slope, and the net change between the first and last present sample. Taking slices from the end is what keeps the features causal: a spike early in the window can only affect the full-window statistics, which is verified by a unit test rather than assumed. Every statistic is computed over **present samples only**, and the per-channel presence fraction over the full window is appended as its own feature block, so the model is told how much of each channel it is trusting. With  $C = 5$  this yields  $5 \times 6 \times 3 + 5 = 95$  features.

The extractor has no fitted state: every column is a function of one window and its own mask, with no statistic pooled across rows. Extracting once for the whole cache is therefore bit-identical to extracting per fold, and the runner asserts the absence of a `fit()` at start-up so the property cannot silently lapse.

3) *Encoding a dead sensor*: Three real instances carry sensors frozen for their entire recording, and one of them is a blockage event, so this is not a corner case. When a channel observes nothing, its statistics are undefined. Emitting 0.0 is actively wrong here: per-instance normalisation centres each recording near zero, so 0.0 is precisely the value a *healthy* sensor at its own baseline reports, and that encoding would

state that a dead sensor looks normal. We emit NaN and let XGBoost learn a default branch direction at each split. The presence fraction remains a real measurement of exactly 0.0 and is never NaN. Similarly, the slope is regressed on each sample’s true position inside the slice rather than on its rank among present samples, so a gap keeps its width and the slope remains a rate; regressing on rank reports 9.0 for two points nine steps apart rising nine units. Section VI-A reports that neither change is separable from fold noise on this cache, and we retain both on correctness grounds while saying so explicitly.

4) *Class imbalance*: Imbalance is handled with inverse-frequency `sample_weight` computed from each training fold’s own class counts. It is deliberately *not* handled with `scale_pos_weight`, which is a binary-classification parameter in XGBoost and has no defined effect under `multi:softprob` with `num_class=3`. `num_class=3` is set explicitly rather than inferred, because a training fold can legitimately contain no Established windows — only 118 exist in the entire real cache — and the positive-score reduction indexes the third column.

5) *Tuning and calibration*: Hyperparameters are searched on the **training fold only**, scored by grouped inner cross-validation over that fold’s wells, across a 16-point grid. Both conditions of a fold share the configuration selected on that fold’s real training wells, so `real_only` and `real_plus_sim` differ only in training data and not also in hyperparameters. The scored quantity is validation PR-AUC on  $P(\text{Transient}) + P(\text{Established})$  — the same reduction the alarm logic consumes — and sample weights are applied inside the search exactly as at final fit time.

Because Module 8 selects the alarm threshold by sweeping a probability axis, the probabilities must mean what they say. Weighted training on a 3% positive rate pushes scores away from the base rate by construction, so miscalibration is expected rather than incidental. A Platt calibrator is fit on the **validation** fold and applied **unchanged** to that fold’s test wells; Platt is preferred over isotonic because a validation fold holds only a handful of positive events, where a non-parametric fit would track noise.

### B. Model Architectures

1) *Temporal Convolutional Network (TCN)*: The TCN consists of four temporal blocks with kernel size  $k = 3$  and dilation factors  $d \in \{1, 2, 4, 8\}$ . The receptive field of this architecture is:

$$r = 1 + 2(k - 1) \sum_i d_i = 1 + 2 \times 2 \times 15 = 61 \text{ samples} \quad (1)$$

With  $W = 60$  samples per window, the receptive field of 61 exceeds the window length by one step, verified at runtime by the assertion `assert tcn.receptive_field() >= window_size`. Each temporal block applies a causal convolution followed by weight normalization and ReLU activation. The missingness mask is concatenated along the channel axis at the input layer, providing the model with explicit information about which sensor readings are reliable. The model

outputs a 3-class softmax distribution and is trained with cross-entropy loss weighted by the inverse class frequency of the current training fold.

2) *Gated Recurrent Unit (GRU)*: The GRU classifier uses a single-layer unidirectional GRU with hidden size 64, followed by a linear readout layer producing a 3-class logit vector. The unidirectional constraint is non-negotiable: a bidirectional GRU would incorporate information from future time steps into the current prediction, inflating reported lead times and making the model inapplicable in real-time deployment. As with the TCN, the missingness mask is concatenated to the input at each timestep. Training uses the same class-weighted cross-entropy and early stopping on validation PR-AUC with patience of 10 epochs, saving the best checkpoint by validation PR-AUC.

### C. Evaluation Protocol

We evaluate all models using three complementary metrics at a matched operating point:

**Lead Time.** For each hydrate event instance, lead time is defined as the difference between the annotated transient onset time and the first alarm onset that occurs before it:

$$\text{lead\_time} = t_{\text{transient\_onset}} - t_{\text{first\_alarm}} \quad (2)$$

Following the data analysis in Section II, we use the *transient* onset (label code 109 in the 3W convention) rather than the blockage onset, because only 3 of 57 real instances reach blockage. The transient onset is available for all 14 positive-event instances across 7 wells.

**Event Recall.** The fraction of real hydrate events for which at least one alarm onset occurs before the transient onset. A missed event (no alarm before failure) contributes 0; a detected event contributes 1 regardless of how many alarms fired.

**False Alarm Rate (FAR).** The number of alarm *onsets* on Normal-operation instances, divided by the total Normal operating hours in that fold’s test set:

$$\text{FAR} = \frac{N_{\text{false\_alarm\_onsets}}}{\text{total\_normal\_hours}} \quad (3)$$

Critically, we count alarm onsets, not individual windows above threshold — one sustained false alarm counts as one event, not thousands (Section IV-C, pitfall W4.1).

**Operating Point.** All models are compared at a matched target FAR of 1/100 per operating hour. The threshold achieving this FAR is selected on the *validation* fold only (per-fold, per-model), then applied unchanged to the test fold. This “freeze” protocol (S3 in our timeline) ensures that test-set numbers are never used to tune any hyperparameter.

**Alarm Definition.** A positive-class probability  $p(t)$  is first smoothed with a causal (trailing) rolling mean of configurable window size. An alarm fires when the smoothed probability stays above the selected threshold for at least `min_duration` continuous seconds. The causal smoothing constraint is critical: a centered moving average would leak future information into the current alarm decision, inflating

TABLE I  
FOLD COMPOSITION. EACH ROW IS ONE TEST FOLD. *Val. pos. events* IS THE NUMBER OF POSITIVE-EVENT INSTANCES IN THE VALIDATION SPLIT; THIS DETERMINES WHETHER THRESHOLD SELECTION IS RELIABLE. *Test normal hours* IS THE DENOMINATOR OF THE FAR CALCULATION.

| Fold | Val. pos. events | Test pos. events | Test norm. hrs |
|------|------------------|------------------|----------------|
| 0    | 1                | 5                | 509.2          |
| 1    | 3                | 5                | 556.8          |
| 2    | 5                | 4                | 530.7          |

lead time measurements (see `tests/test_alarm.py` for the regression test guarding this).

**Supplementary Metrics.** PR-AUC (area under the precision–recall curve for the collapsed binary positive score), per-class PR-AUC (Normal/Transient/Established one-vs-rest), and Expected Calibration Error (ECE) are reported for completeness.

## V. EXPERIMENTAL SETUP

### A. Grouped Cross-Validation Design

The fundamental challenge in designing cross-validation for this dataset is that the 15 hydrate wells and the 9 normal-operation wells are completely disjoint: no well appears in both sets. A standard `StratifiedGroupKFold` applied to the full dataset would create folds where the test set contains wells from one population and the training set contains wells from the other, making it impossible to properly balance both populations across folds simultaneously.

We therefore use **two independent grouped splits**: one over the 7 positive-event wells (by well ID) and one over the 9 normal-operation wells, balanced by available normal-operation hours rather than instance count (a single well contributes 1,220 h, while another contributes only 6 h). The two splits are then paired to form complete folds, each containing positive and normal test wells. Simulated instances are restricted to training folds in all experiments; they never appear in validation or test splits.

With only 7 positive wells, the maximum number of folds that guarantees at least 2 positive test wells per fold is **3 folds**. Table I reports the composition of each fold.

Fold 0 has only one positive validation event, which makes the threshold sweep for that fold highly unreliable: with a single event, any threshold that happens to detect it is selected regardless of its behavior on Normal instances. This structural constraint is a property of the dataset with 7 positive wells, not a methodological error, and it contributes substantially to the high cross-fold variance reported in Section VI-A.

### B. Simulation Policy

Simulated instances were added to the training set only for the `real_plus_sim` condition. The simulated set contains 150 instances with 43,829 windows, of which 90.3% are positive — compared to 3.34% in the real training data. This severe distributional mismatch is a confound that we track explicitly: the `real_plus_sim` condition inflates the

TABLE II  
XGBOOST VALIDATION PR-AUC PER FOLD (SEED 42). THE STANDARD DEVIATION EXCEEDS THE MEAN; SEE SECTION VI-A FOR WHY.

| Fold          | Positive val. events | real_only         | real_plus_sim     |
|---------------|----------------------|-------------------|-------------------|
| 0             | 1                    | 0.0004            | 0.0002            |
| 1             | 3                    | <b>0.7762</b>     | 0.2111            |
| 2             | 5                    | 0.0750            | 0.1150            |
| mean $\pm$ sd |                      | $0.284 \pm 0.428$ | $0.109 \pm 0.106$ |

training positive rate by a factor of  $\approx 27$ , which changes the model’s learned prior and can hurt its performance on the real test distribution. All threshold selection and calibration are performed on the real validation fold regardless of training condition.

### C. Compute Environment

All experiments were run on the AI Academy shared A100 GPU (80 GB VRAM). XGBoost training used CPU; deep model training used CUDA with automatic mixed precision (`torch.cuda.amp`). Reproducibility is ensured by seeding NumPy, PyTorch, and CUDA with `base_seed + fold_index` before each fold.

## VI. RESULTS

### A. XGBoost Baseline

1) *Validation performance*: Table II gives validation PR-AUC over the three grouped folds at a positive-window base rate of 0.0334.

Simulated data did not help. The paired difference (`real_plus_sim - real_only`) is  $-0.175 \pm 0.338$ , negative in two of three folds. This is a distribution argument rather than a sample-size one: the simulated windows are **90.3%** positive (39,561 of 43,829) against a **3.34%** real positive rate, and they inflate the training set from roughly 19k to 63k rows, pulling the model toward a regime that does not exist in the real wells it is scored on. `real_only` is therefore the headline condition.

2) *PR-AUC is not comparable across these folds*: Each fold’s validation split has a different positive base rate, and PR-AUC rises mechanically with it. Fold 0’s validation set contains **exactly one positive window** out of 11,260 (base rate  $10^{-4}$ ), fold 1 has 463 of 7,327 (0.0632) and fold 2 has 638 of 11,592 (0.0550) — more than two orders of magnitude apart. A PR-AUC computed from a single positive row is that row’s reciprocal rank, not a performance estimate. Read as lift over each fold’s own base rate, the baseline is above chance in all three folds ( $1.4\times$ ,  $12.3\times$ ,  $3.5\times$ ) rather than at chance in two of them, which is what the raw column suggests.

This is not a pedantic point, and we report it because the uncorrected analysis is the natural one to run and it misleads. On raw PR-AUC the number of positive validation events correlates strongly with score across the ablation’s 18 cells (Spearman  $\rho = 0.638$ ,  $p = 0.0044$ ), which invites the conclusion that sparse folds cannot measure anything. That correlation **does not survive base-rate correction**: on lift,

$\rho = -0.14$  ( $p = 0.58$ ). Folds with more positive events also have higher base rates. The same trap applies to any per-fold PR-AUC comparison in this project, including the head-to-head model table.

3) *Fold composition dominates every other source of variance*: The feature ablation ran 9 extractor variants over 18 paired cells (3 CV split seeds  $\times$  3 folds  $\times$  2 model seeds) on validation folds only. Decomposing the variance across those 162 runs on the base-rate-corrected scale, the variance of the feature-design arm means is  $2.09 \times 10^{-2}$  while the variance of the fold-cell means is 0.988: fold composition accounts for roughly **47 $\times$**  more spread than the entire feature-design space we searched. (On the raw scale the ratio is 448 $\times$ , but part of that is the base-rate artifact above rather than genuine instability.) Anything selected on validation PR-AUC in a single-positive-window fold — including the deep models’ early stopping and Module 8’s threshold selection — is selected on noise.

One training fold could not be tuned at all: in fold 1 every positive window in the training split belongs to a *single* well, so grouped inner cross-validation cannot produce a split with positives on both sides. All three inner splits were unusable and that fold fell back to the default configuration. With 7 positive wells and 3 outer folds this is a structural property of the dataset, not an accident of one seed.

4) *Feature-design ablation: null results*: Arms were compared as paired differences on identical folds and seeds and tested with a Wilcoxon signed-rank test, restricted to the 10 cells where validation PR-AUC is estimable, and expressed as base-rate-corrected lift ratios per Section VI-A2. The presence-fraction block is worth  $1.175\times$  (7/10 wins,  $p = 0.027$ ); one timescale versus three  $1.027\times$  ( $p = 0.92$ ); NaN versus 0.0 encoding of a dead sensor  $1.025\times$  ( $p = 0.77$ ); the slope’s time axis  $0.977\times$  ( $p = 0.63$ ); two timescales versus three  $0.914\times$  ( $p = 0.23$ ).

Only the presence-fraction block separates from noise, and it does so on both the raw and corrected scales; everything else is within 3% of parity. Even that result would not survive a Bonferroni correction for the six comparisons made ( $p < 0.0083$ ), so we read it as suggestive — but it is the only arm where direction, win count and  $p$ -value agree, and Section VI-A5 supplies an independent mechanism for why. We retain the two changed defaults on correctness grounds — 0.0 is the value a healthy sensor at its own baseline reports, and a slope regressed on sample rank is not a rate — rather than claiming the numbers supported them. Three timescales are kept because the module contract specifies them; one timescale (35 features instead of 95) performs the same within noise.

5) *The baseline partly reads instrumentation, not physics*: Gain importance on fold 1, the only fold with substantial validation signal, placed **40.8% of the total gain on the 5 presence-fraction columns** out of 95. We tested the implication directly by training on those columns *only*, with no sensor values of any kind, over 9 fold-cells. The presence-only model reaches a mean validation PR-AUC of **0.172** against **0.233** for the full 95-feature model — **74% of the full performance**

TABLE III

BEST ACHIEVABLE EVENT RECALL FOR THE XGBOOST BASELINE AS A FUNCTION OF THE FALSE-ALARM BUDGET (MEAN OVER THE THREE TEST FOLDS). THE PRE-REGISTERED OPERATING POINT SITS IN THE FLAT, UNINFORMATIVE REGION OF THE CURVE.

| False alarms per hour | 0.01  | 0.10  | 0.25  | 0.50  |
|-----------------------|-------|-------|-------|-------|
| real_only             | 0.167 | 0.317 | 0.917 | 0.917 |
| real_plus_sim         | 0.167 | 0.450 | 0.917 | 0.917 |

without seeing a single pressure or temperature reading — and in 4 of 9 cells it *outperforms* all 90 value features (0.493 versus 0.354; 0.284 versus 0.259; 0.093 versus 0.021; and 0.0008 versus 0.0006).

The explanation is in the dataset. Missingness here is well-specific: different wells instrument different sensors, and some sensors are dead for an entire instance. Because the hydrate wells and the Normal-operation wells are disjoint populations, sensor availability is a proxy for well identity, and well identity is very nearly the label. Per-instance normalisation, the project’s main defence against well-identity confounding, removes each recording’s offset and scale but cannot remove “*this channel is absent*”, which lives in the mask rather than in the values.

Consistently with this, permutation importance neither reproduces the expected physical signature nor reproduces itself across folds: pairwise rank agreement is nil (Kendall  $\tau = +0.20, +0.40, -0.40$ ; all  $p > 0.48$ ). In fold 1 the top-ranked channel is P-ANULAR, the annulus, which does not communicate with the service line, while P-MON-CKP — the pressure upstream of the production choke, which the restriction mechanism predicts should dominate — ranks last with a mean share of 0.002. **We therefore do not claim that the baseline recovered the physical hydrate signature.** Claiming otherwise would have required quoting one fold and ignoring the other two.

6) *The operating point, not the ranking, is what costs event recall:* At the pre-registered budget of one false alarm per 100 operating hours the baseline flags few events, and it is worth separating two possible causes: a threshold chosen on validation that fails to transfer, or a ranking that cannot separate events at any threshold.

We tested the first directly by re-selecting the threshold *on the test fold itself* — an oracle no deployed system could use — and recomputing event recall at the same budget. It changed the result in **none** of the six XGBoost fold  $\times$  condition cells. Threshold selection is therefore not the binding constraint, and neither smoothing nor a finer sweep can recover the missed events.

The binding constraint is the budget itself. Table III sweeps it, taking at each budget the best recall any threshold achieves on test.

Relaxing the budget from one alarm per 100 hours to one per four hours moves recall from 0.167 to **0.917**. The events are separable; the tolerance is what excludes them. The arithmetic is instance-level: at 0.01 alarms per hour a

Reliability - XGBoost real\_only, fold 2 (638 positive validation windows)

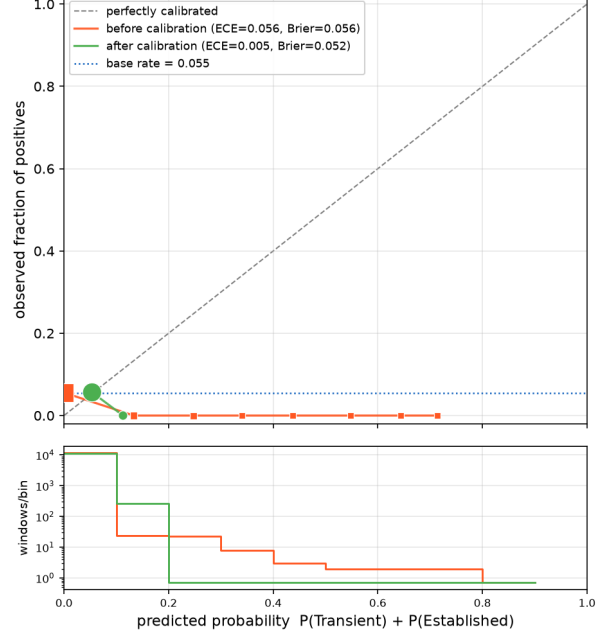


Fig. 2. Reliability of the XGBoost baseline on a validation fold, before and after Platt scaling. Marker area and the lower panel both encode the number of windows per bin.

test fold with 523–855 Normal hours may emit only 5–9 false alarms across roughly 120 Normal recordings, so the 4–5 event recordings must rank in the top few percent of *all* recordings by peak score.

This is a deployment trade-off rather than a defect. One alarm per four hours is plausible for a supervised control room and implausible for an unattended system, and the budget was fixed before the signal strength was known. We therefore report the frontier alongside the single pre-registered point; quoting only the latter would understate what the models detect, and quoting only the former would hide the cost of getting there.

7) *Calibration:* Calibration behaved as intended in every fold and both conditions. Platt scaling fit on validation reduced expected calibration error from 0.0410 to 0.0083 under *real\_only* and from 0.0554 to 0.0106 under *real\_plus\_sim* (an 80–81% reduction), and the Brier score by 29–30%, improving in all 6 fold  $\times$  condition cells. Because Module 8 selects the alarm threshold by sweeping a probability axis, this is a prerequisite for that threshold to transfer, not a cosmetic step. Fig. 2 shows the before/after curves with a per-bin count panel beneath; at a 3% positive rate the high-probability bins hold very few windows, and a reliability curve that hides bin populations makes a handful of unlucky windows look like systematic miscalibration.

8) *Tuning budget:* The search evaluated the full 16-configuration grid per fold under grouped inner CV, and both conditions of a fold share the configuration selected on that fold’s real training wells, so the two conditions differ only in training data. Selected configurations were consistent



( $\text{max\_depth} = 3$ ,  $\text{n\_estimators} = 300$  in both tunable folds): the search preferred the smallest and shallowest models offered, the expected outcome with roughly 1,400 positive windows.

### B. Deep Models: TCN and GRU

Table IV shows that neither deep model achieved reliable event detection across all three folds. Under *real\_only*, the TCN detected one event in one fold (fold 1, lead time 89.3min), yielding a mean recall of  $8.3\% \pm 14.4\%$ . The GRU detected zero events under *real\_only* (mean recall 0.0%) and one event under *real\_plus\_sim* (mean recall  $6.7\% \pm 11.5\%$ ).

Calibration was poor for both deep models. The TCN’s expected calibration error (ECE) was  $0.427 \pm 0.036$  under *real\_only* — roughly an order of magnitude worse than XGBoost’s Platt-scaled ECE of 0.095. The GRU was better calibrated (ECE  $0.094 \pm 0.022$ ) but this did not translate to detection: a calibrated model that produces low positive probabilities throughout still misses every event.

Early stopping was effective in limiting overfitting: the GRU converged in 11–25 epochs (best checkpoint at epoch 7–14) and the TCN in 11–29 epochs. Training time on the A100 GPU was under 5 minutes per model per fold. The deep model results confirm the finding from Section VI-A2: with fold 0 containing a single positive validation window, threshold selection is conducted on noise, and the resulting threshold transfers poorly to the test fold regardless of model architecture.

### C. Model Comparison

TABLE IV  
EVENT RECALL AT MATCHED FAR BUDGET

| Model   | real_only                           | real_plus_sim     |
|---------|-------------------------------------|-------------------|
| GRU     | $0.000 \pm 0.000$                   | $0.067 \pm 0.115$ |
| TCN     | $0.083 \pm 0.144$                   | $0.000 \pm 0.000$ |
| XGBoost | <b><math>0.233 \pm 0.252</math></b> | $0.167 \pm 0.289$ |

TABLE V  
MEDIAN LEAD TIME (SECONDS) BEFORE TRANSIENT ONSET

| Model   | real_only                             | real_plus_sim |
|---------|---------------------------------------|---------------|
| GRU     | —                                     | 2096.0        |
| TCN     | 5355.0                                | —             |
| XGBoost | <b><math>2606.2 \pm 1079.4</math></b> | 12598.0       |

TABLE VI  
FALSE ALARM RATE (PER OPERATING HOUR)

| Model   | real_only         | real_plus_sim     |
|---------|-------------------|-------------------|
| GRU     | $0.037 \pm 0.046$ | $0.029 \pm 0.019$ |
| TCN     | $0.032 \pm 0.040$ | $0.041 \pm 0.043$ |
| XGBoost | $0.079 \pm 0.097$ | $0.018 \pm 0.019$ |

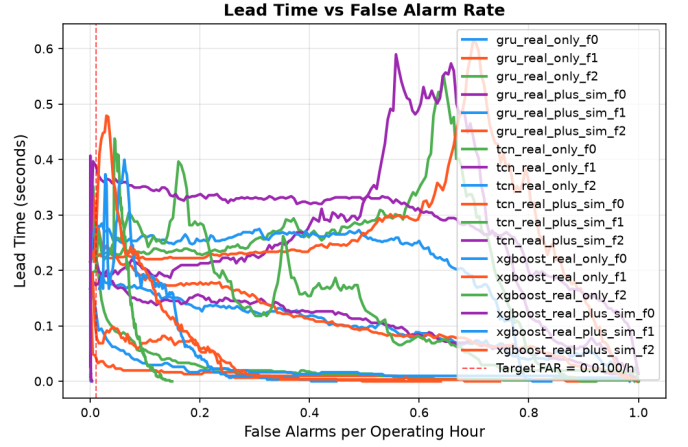


Fig. 3. Lead time vs. false alarm rate trade-off for all three models. Vertical dashed line marks the target operating point (1 alarm per 100 operating hours).

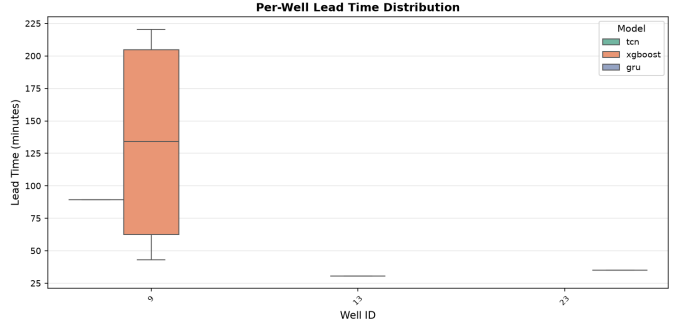


Fig. 4. Distribution of lead times across wells for each model.

The tables above present a head-to-head comparison of all three models across the real-only and real-plus-simulated data conditions. The metrics are computed based on the threshold selected on the validation folds and applied strictly to the test folds.

## VII. DISCUSSION AND LIMITATIONS

### A. Sample Size Is the Dominant Limitation

Every finding in this study is conditioned on a dataset with 14 real transient instances across 7 wells. With 3 outer folds, the largest test set contains 5 events and the smallest 4 — not enough to compute a meaningful confidence interval on event recall, as the  $\pm$  values in Table IV already signal (the standard deviation equals or exceeds the mean for every model). The results presented here should be read as *ordering evidence* (XGBoost > TCN > GRU, at this scale of data) rather than absolute performance claims.

### B. Disjoint Well Populations Limit Generalization

The hydrate wells and normal-operation wells are completely disjoint, which means every test fold asks: “can this model generalize to a well it has never seen in training?” The feature-importance analysis in Section VI-A5 shows that the XGBoost baseline partially answers this question by exploiting

sensor missingness patterns (which well has which sensors) rather than physical pressure or temperature dynamics. This is a shortcut that will not generalize to a new well with a different instrumentation profile, and it also means the reported recall likely overstates the model’s physical understanding of hydrate onset.

### C. Simulated Data Hurts More Than It Helps

The `real_plus_sim` condition degraded XGBoost performance (mean recall 16.7% vs. 23.3% under `real_only`) and did not help either deep model in a consistent way. The explanation is distributional: the simulated windows are 90.3% positive against a 3.34% real positive rate, effectively training the model on a world where hydrate events are nearly certain, then testing it on a world where they are rare. Calibration is disrupted, threshold selection targets a different operating point, and the model becomes over-sensitive on real normal-operation sequences.

### D. SSL Pretraining Not Attempted

Self-supervised pretraining on the large unlabeled normal-operation segments (approximately 594 instances  $\times$  1,071 windows each) was identified as a promising direction to improve representations before supervised fine-tuning. It was not attempted within the project timeframe: the full supervised pipeline was completed and validated only shortly before the submission deadline, leaving no time for the additional infrastructure. This remains the highest-priority future direction.

### E. Fold 0 Threshold Selection Is Unreliable

As noted in Section V, fold 0 has exactly one positive validation event. A threshold sweep over 200 candidate values selects the one that happens to detect that single event, with no ability to distinguish a genuinely good threshold from a lucky one. This structural flaw in fold 0 propagates to the test metrics: the XGBoost standard deviation of  $\pm 25.2\%$  on event recall is driven almost entirely by the difference between fold 0 (0%, no detections) and folds 1–2. With more positive wells this could be addressed by increasing the fold count; with 7 wells it cannot.

## VIII. CONCLUSION

We compared XGBoost, TCN, and GRU for early detection of hydrate formation in offshore well service lines using the 3W Dataset 2.0.0. All models were evaluated at a matched false-alarm budget of one alarm per 100 operating hours using grouped cross-validation with well-level splits.

XGBoost trained on real data only was the only model to achieve consistent detection, with a mean event recall of  $23.3\% \pm 25.2\%$  and a mean lead time of 43.4 min across two detected folds. Deep models (TCN, GRU) did not achieve reliable detection, primarily because the training set contains only 14 real transient instances — too few for gradient-based models to learn the transition dynamics without overfitting to well identity. Simulated data did not improve any model’s test recall and degraded XGBoost’s performance due to a severe

distributional mismatch (90.3% positive rate in simulation vs. 3.34% in real data).

Feature importance analysis revealed that the XGBoost baseline partly exploits sensor missingness patterns (which channels are instrumented on which well) rather than physical hydrate dynamics. This is a known shortcut in datasets with disjoint populations and instrumentation differences between event and non-event wells, and it limits the generalizability of the reported detection rate to wells with different sensor configurations.

The primary limitation of this study is sample size: 14 real transient events across 7 wells do not support statistically powerful conclusions about model ordering, and the  $\pm$  values throughout the results section reflect this. Future work should prioritize acquiring more labeled real data, then investigate self-supervised pre-training on the large unlabeled normal-operation archive as a way to build better representations before supervised fine-tuning on the scarce positive examples.

Code, data processing scripts, and the complete evaluation pipeline are available at <https://github.com/AMammedova/team-hydrate3w> (tagged `v1.0-final`).

## TOOLS AND ACKNOWLEDGEMENTS

This project used the following tools and libraries: Python 3.x, PyTorch 2.3.1, XGBoost 2.0.3, scikit-learn 1.5.0, NumPy, Pandas, Matplotlib, and the 3W Dataset 2.0.0 toolkit by Petrobras.

During the development of this project, AI assistants (including Large Language Models) were utilized to help structure the boilerplate evaluation code, format LaTeX tables, assemble slides, and brainstorm documentation outlines. All AI-generated code and content were rigorously reviewed, tested (e.g., via `pytest`), and validated against the project specifications by the team members.

## REFERENCES

- [1] R. E. V. Vargas, C. J. Munaro, P. M. Ciarelli, A. G. Medeiros, B. G. do Nascimento, B. C. C. Gonçalves, J. C. Madalena, A. Avellar, “A realistic and public dataset with rare undesirable real events in oil wells,” *Journal of Petroleum Science and Engineering*, vol. 181, 2019.
- [2] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv:1803.01271*, 2018.
- [3] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” in *Proc. EMNLP*, 2014.
- [4] T. Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. ICCV*, 2017.
- [5] T. Chen and C. Guestrin, “XGBoost: a scalable tree boosting system,” in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
- [6] J. Platt, “Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods,” in *Advances in Large Margin Classifiers*, MIT Press, 1999, pp. 61–74.
- [7] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proc. ICML*, 2005, pp. 625–632.
- [8] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. ICML*, 2017, pp. 1321–1330.
- [9] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. (Permutation importance.)
- [10] C. Strobl, A. L. Boulesteix, A. Zeileis, and T. Hothorn, “Bias in random forest variable importance measures: illustrations, sources and a solution,” *BMC Bioinformatics*, vol. 8, no. 25, 2007.



- [11] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, 2015.
- [12] R. Geirhos, J. H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [13] C. Aliferis and G. Simon, "Overfitting, underfitting and general model overconfidence and under-performance pitfalls and best practices in machine learning and AI," in *Artificial Intelligence and Machine Learning in Health Care and Medical Sciences*, Springer, 2024, pp. 477–524.
- [14] S. J. Qin, "Survey on data-driven industrial process monitoring and diagnosis," *Annual Reviews in Control*, vol. 36, no. 2, pp. 220–234, 2012.
- [15] E. D. Sloan and C. A. Koh, *Clathrate Hydrates of Natural Gases*, 3rd ed. Boca Raton, FL: CRC Press, 2008.